

Project: Automated EBS Snapshot and Notification System

Services Used: EC2, EBS, Lambda, SNS, EventBridge, CloudWatch

Description: This project automates the creation of EBS snapshots for a specific EC2 instance using AWS Lambda. It also deletes old snapshots based on a retention policy and sends email notifications using SNS after each execution.

Definition: The Automated EBS Snapshot and Notification System is designed to provide automated backups of EC2 instance volumes. It uses AWS Lambda to create snapshots, manage retention, and Amazon SNS to notify users. EventBridge schedules the execution, ensuring continuous and automatic backups.

Scope of the Project:

- Automated snapshot creation for EC2 volumes
- Retention policy (keeps latest snapshots, deletes old ones)
- Email notification system using SNS
- Fully automated backup using EventBridge scheduling

This Project Covers:

- **Automated EBS Snapshot Creation:** Automatically creates backups of EBS volumes attached to the specified EC2 instance.
- **Snapshot Retention Management:** Deletes older snapshots based on the defined retention count to optimize storage and reduce cost.
- **Scheduled Automation:** Backups are triggered at regular intervals using EventBridge (CloudWatch Events), eliminating manual intervention.

Methodology:

1. Lambda + EventBridge (Implemented Method)

- Uses AWS services: EC2, EBS, Lambda, EventBridge, SNS
- Fully automated backup process
- Creates and deletes snapshots based on retention policy
- Sends notifications using SNS
- Serverless and cost-effective solution

2. EC2 + CRON + AWS CLI (Alternative Method)

- Uses EC2 instance, CRON scheduler, and AWS CLI
- CRON is a time-based scheduler used in Linux to run tasks automatically at specific times.
- Automates backups using scripts and cron jobs
- Requires a continuously running EC2 instance
- Higher operational cost and maintenance
- *This method was not implemented in this project but is an alternative approach*

AWS Services Used in the Project:

1. Amazon EC2 (Elastic Compute Cloud)

Definition: Amazon EC2 is a web service that provides resizable virtual servers in the cloud, allowing users to run applications and workloads on demand.

Purpose: Provides virtual servers (instances) where applications run and EBS volumes are attached.

Role: The EC2 instance acts as the source, and its attached EBS volumes are backed up using snapshots.

2. Amazon EBS (Elastic Block Store)

Definition: Amazon EBS is a managed block storage service designed to be used with EC2 instances, offering persistent and reliable storage.

Purpose: Provides persistent block storage for EC2 instances.

Role: Volumes are backed up by taking snapshots, which are created and managed automatically.

3. AWS Lambda

Definition: AWS Lambda is a serverless compute service that allows users to run code without provisioning or managing servers.

Purpose: Runs serverless code to automate backup operations.

Role: Fetches EC2 instance volumes, creates snapshots, deletes old snapshots based on retention policy, and sends notifications.

4. Amazon SNS (Simple Notification Service)

Definition: Amazon SNS is a messaging service used to send notifications and alerts via email or other protocols.

Purpose: Sends real-time notifications to users.

Role: Sends email alerts after snapshot creation and deletion, including success and failure messages.

5. Amazon EventBridge (CloudWatch Events)

Definition: Amazon EventBridge is a service that delivers a near real-time stream of system events and enables automated actions based on schedules or events.

Purpose: Schedules and triggers automated tasks.

Role: Triggers the Lambda function at regular intervals using a cron expression to automate snapshot creation and deletion.

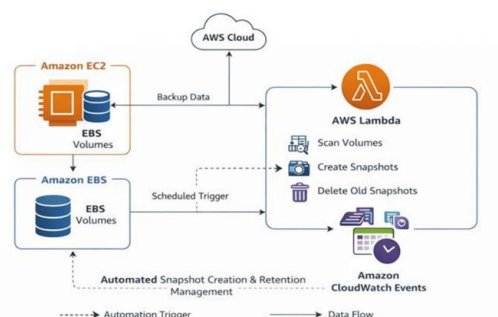
6. Amazon CloudWatch

Definition: Amazon CloudWatch is a monitoring and logging service for AWS resources and applications.

Purpose: Monitors performance and stores logs.

Role: Stores Lambda execution logs, tracks when backups are performed, and helps in troubleshooting errors.

Architecture Flow: EC2 → EBS → Lambda → SNS → EventBridge → CloudWatch

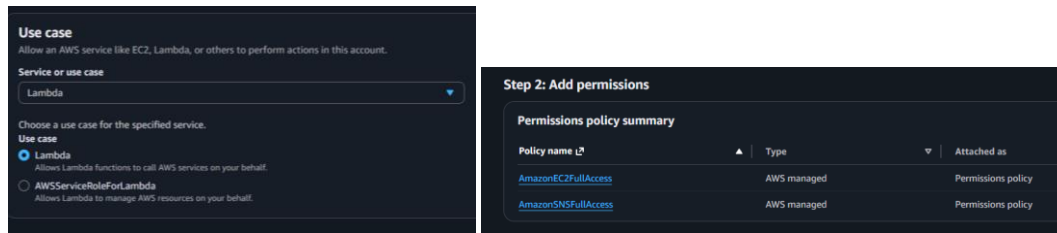


Implementation Steps

Step 1: Create IAM Role

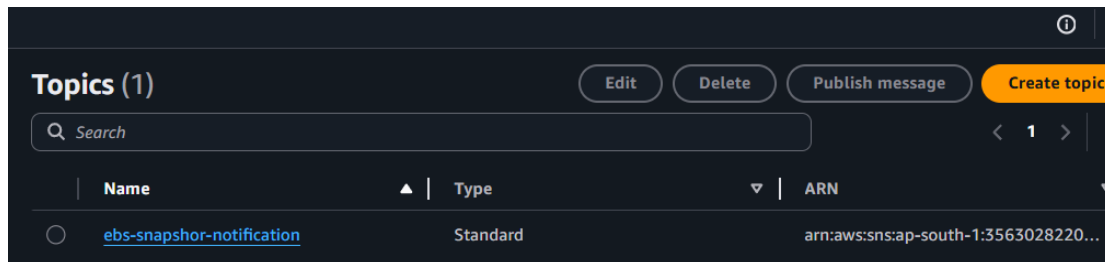
Created an IAM role for Lambda with the following permissions:

- **AmazonEC2FullAccess**
 - **AmazonSNSFullAccess**
- Role name: **ec2-ebs-lambda-role**



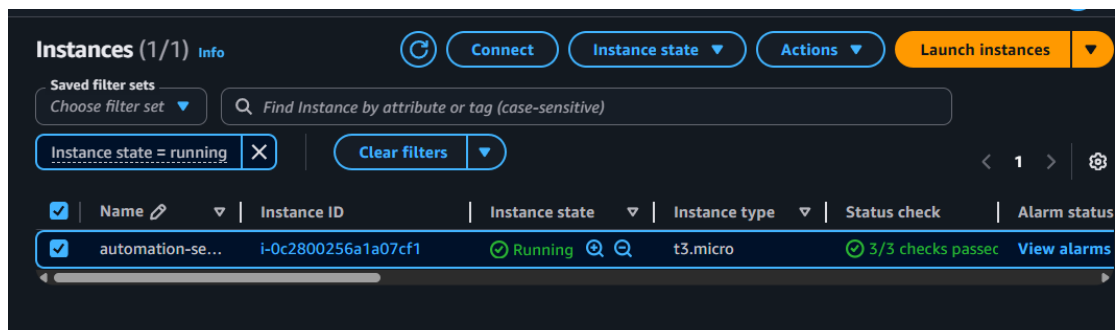
Step 2: Create SNS Topic:

Created an SNS topic named **ebs-snapshot-notification** and added an **email subscription** to receive backup notifications.

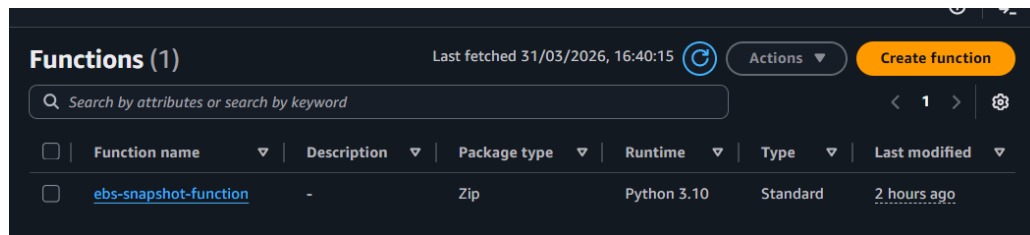


Step 3: Create EC2 Instance

Created an EC2 instance named **automation-server**.
This instance is the source whose attached EBS volume snapshots are taken automatically.



Step: Create Lambda Function



- Go to AWS Console → Lambda → Click **Create Function**
- Select **Author from Scratch**
- Enter Function Name: **ebs-snapshot-function**
- Choose Runtime: **Python 3.10**
- Select Execution Role → Choose existing role → **ec2-ebs-lambda-role**
- Click **Create Function**
- Go to **Code section** → Paste the Python Lambda code deploy it.

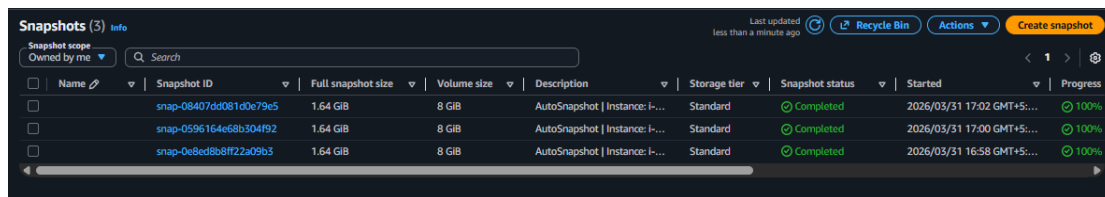
```
lambda_function.py
1 import boto3
2 import datetime
3
4 ec2 = boto3.client('ec2')
5 sns = boto3.client('sns')
6
7 SNS_TOPIC_ARN = "arn:aws:sns:ap-south-1:356382822046:ebs-snapshot-notification"
8 INSTANCE_ID = "i-8c2806256a1a07cfl"
9 RETENTION_COUNT = 3
10
11 def lambda_handler(event, context):
12     try:
13         instance_details = ec2.describe_instances(InstanceIds=[INSTANCE_ID])
14         block_devices = instance_details['Reservations'][0]['Instances'][0]['BlockDeviceMappings']
15
16         snapshots_created = []
17         snapshots_deleted = []
18
19         for block in block_devices:
20             volume_id = block['Ebs']['VolumeId']
21
22             description = (
23                 f"AutoSnapshot | Instance: {INSTANCE_ID} | Volume: {volume_id} | "
24                 f"{datetime.datetime.utcnow().strftime('%Y-%m-%d_%H-%M-%S')}"
25             )
26
27             snapshot = ec2.create_snapshot(
28                 VolumeId=volume_id,
29                 Description=description
30             )
31
32             snapshot_id = snapshot['SnapshotId']
33             snapshots_created.append(snapshot_id)
34
35             ec2.create_tags(
36                 Resources=[snapshot_id],
37                 Tags=[
38                     {'Key': 'CreatedBy', 'Value': 'LambdaAutoSnapshot'},
39                     {'Key': 'InstanceId', 'Value': INSTANCE_ID},
40                     {'Key': 'VolumeId', 'Value': volume_id}
41                 ]
42             )
43
44         lambda_snapshots = ec2.describe_snapshots(
45             OwnerIds=['self'],
46             Filters=[
47                 {'Name': 'volume-id', 'Values': [volume_id]},
48                 {'Name': 'tag:CreatedBy', 'Values': ['LambdaAutoSnapshot']}
49             ])['Snapshots']
50
51         sorted_snaps = sorted(lambda_snapshots, key=lambda x: x['StartTime'], reverse=True)
52
53         if len(sorted_snaps) > RETENTION_COUNT:
54             old_snaps = sorted_snaps[RETENTION_COUNT:]
55             for old in old_snaps:
56                 ec2.delete_snapshot(SnapshotId=old['SnapshotId'])
57                 snapshots_deleted.append(old['SnapshotId'])
58
59         message = {
60             f"Snapshot Automation for Instance {INSTANCE_ID}\n\n"
61             f"Snapshots Created ({len(snapshots_created)}): {snapshots_created}\n"
62             f"Snapshots Deleted ({len(snapshots_deleted)}): {snapshots_deleted}\n"
63         }
64
65         sns.publish(
66             TopicArn=SNS_TOPIC_ARN,
67             Subject=f"EC2 Snapshot Report - Instance {INSTANCE_ID}",
68             Message=message
69         )
70
71         return {"status": "success"}
72
73     except Exception as error:
74         sns.publish(
75             TopicArn=SNS_TOPIC_ARN,
76             Subject=f"Snapshot Automation FAILED - Instance {INSTANCE_ID}",
77             Message=str(error)
78         )
79         raise error
```

- Update **INSTANCE_ID** and **SNS_TOPIC_ARN** in the code
- Go to **Configuration** → **General Settings** → **Edit** → Increase Timeout (e.g., 5 minutes) & give execution role (ebs-snapshot-lambda-role).
- Click **Deploy**
- Go to **Test** → **Create Test Event** → Give name (test1)
- Click **Test** to run the Lambda function
- Verify output and check snapshots in **EBS section**

What this code does: This Lambda function automatically creates snapshots of EBS volumes attached to a specific EC2 instance, keeps only the latest 3 snapshots, deletes older ones, and sends email notifications using SNS. It ensures automated backup, retention management, and alerting, and also sends a failure message if any error occurs.

Step 5: Verify Snapshot Creation:

After testing the Lambda function. Go to the **EBS Snapshots** section and verified that snapshots were created successfully.



It deleted the old snapshot.

Step 6: Create EventBridge Schedule Rule

Went to **Amazon EventBridge** and created a **schedule rule**.

- Give name : **eks-lambda-trigger-rule**
- Selected **Schedule Pattern** choose (recurring schedule).
- Schedule type [cron based schedule] **Cron Expression**

Schedule type
Choose the schedule type that best meets your needs.

☒ **Cron-based schedule**
A schedule set using a cron expression that runs at a specific time, such as 8:00 a.m. PST on the first Monday of every month.

☐ **Rate-based schedule**
A schedule that runs at a regular rate, such as every 10 minutes.

Cron expression Info
Define the cron expression for the schedule

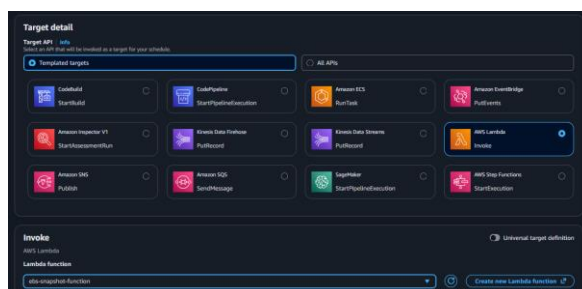
cron ()

Minutes Hours Day of month Month Day of week Year

Flexible time window

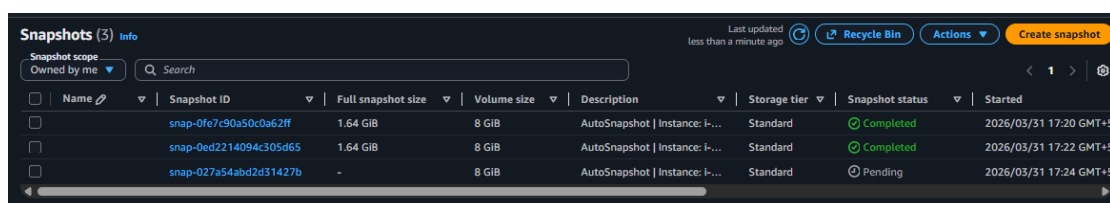
EventBridge triggers Lambda, and Lambda runs the code which creates EBS snapshots.

- Set the target API as **Lambda function**



This configuration makes EventBridge invoke my Lambda function automatically, which performs snapshot creation, deletion, and notification.

- Chose **Invoke AWS Lambda function**
- Execution role (create new execution role)
- Permissions were created automatically for EventBridge to invoke Lambda



Step 7: Monitor the Function

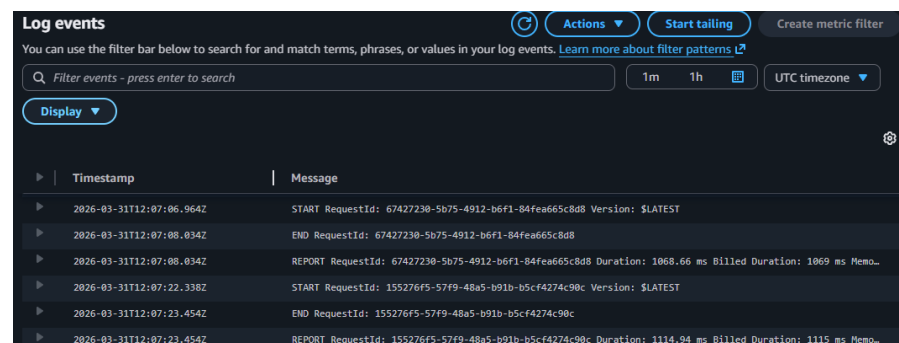
Go to the **Monitoring** section and checked Lambda execution details and logs in **CloudWatch** to confirm when backups were taken and whether the function ran successfully.

Step 1: Go to IAM → Roles → ec2-ebs-lambda-role

Step 2: Click Add permissions.

Step 3: Attach this policy:AWSLambdaBasicExecutionRole

- Go to **Lambda** → **ebs-snapshot-function**
- Click **Test** (run it once)
- Go to **CloudWatch** → **Log groups**
- Now you will see:



Lambda Function Logic

- Fetch EC2 instance details
- Get attached EBS volumes
- Create snapshot for each volume
- Tag snapshots
- Keep latest 3 snapshots
- Delete older snapshots
- Send SNS notification
- Handle errors and send failure alert

Troubleshooting

Issue 1: CloudWatch Logs Not Visible

Solution: Attach AWSLambdaBasicExecutionRole to Lambda so it can create and write logs

Issue 2: Snapshots Not Creating

Solution: Check correct INSTANCE_ID, ensure IAM role has EC2 permissions, and verify Lambda execution in CloudWatch logs

Issue 2: Lambda Timeout Error

Solution: Increase Lambda timeout in Configuration → General Settings.

Conclusion

This project provides a fully automated backup solution for EC2 volumes. It reduces manual effort, ensures data safety, and provides real-time notifications. The system is scalable, cost-effective, and suitable for real-world deployment.